

@qa_insights

Python

Libraries Notes



Essential
Libraries



Real-world
Use Cases



Boost
Productivity



Interview
Ready

— @qa_insights | Learn Python, Build Better Software! —

Introduction to Python Libraries



★ 1. What is a Python Library?

- A collection of pre-written code to solve specific problems.
- Helps developers build powerful applications faster.
- Example: `numpy`, `pandas`, `requests`.

★ 2. Module vs Package

- **Module**: Single `.py` file with functions/classes.
- **Package**: Folder containing modules and an `__init__.py` file.
- Packages help organize large projects.

★ 3. Standard vs Third-party Libraries

- **Standard**: Built into Python (e.g., `os`, `sys`)
- **Third-party**: Available on PyPI and installed via `pip`.
- Third-party libraries extend Python's capabilities.

★ 4. PyPI, pip & Virtual Environment



PyPI: Python Package Index – official repository of Python packages.



pip: Package installer for Python.



Install: `pip install package_name`



venv: Isolated environment for project dependencies.

Python Libraries

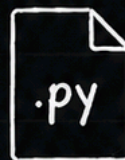
Built-in

Comes with Python
(e.g., `os`, `math`,
`datetime`)

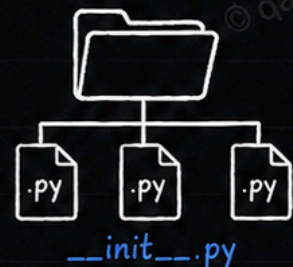
Third-party

Installed via `pip`
(e.g., `pandas`,
`requests`, `flask`)

Module



Package



Why Use Libraries?

- ✓ Save time and effort
- ✓ Reliable and well tested
- ✓ Focus on business logic
- ✓ Boost productivity

Example

```
# Install a library
pip install pandas

# Import the library
import pandas as pd
```

Standard Libraries

★ What?

- Standard libraries are **built-in modules** that come with Python and require no installation.

✓ Why Use Them?

-  Ready to use, no external install
-  Reliable, well tested and secure
-  Improve productivity and save time
-  Cover a wide range of system and application needs

Commonly Used Modules

- os** - Interact with OS 
- sys** - System specific params 
- pathlib** - Object oriented paths 
- shutil** - High level file operations 
- glob** - Pattern based file search 
- subprocess** - Run external commands 
- logging** - Logging and debugging 

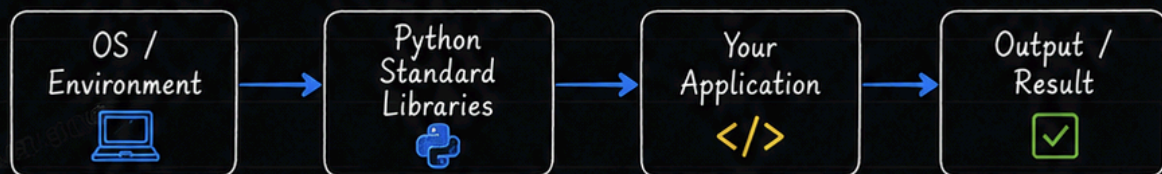
What They Help With?

- File & directory operations
- System and environment info
- Process management
- Searching and handling files
- Logging and debugging

→ Example: Using os Library

```
import os
print(os.getcwd())      # current directory
print(os.listdir('.'))  # list files & folders
os.mkdir('New_Folder')  # create new folder
```

How They Work Together



Always **prefer standard libraries** before using third-party packages.

Interview Tip

Standard libraries are part of Python distribution - **no need to install**.

Data Visualization Libraries



★ What?

- Libraries that help us create beautiful, informative and interactive visualizations.
- Turn data into insights using charts, graphs and dashboards.

📖 Popular Data Visualization Libraries

- **Matplotlib**  Foundation library for static plots and customization.
- **Seaborn**  Built on Matplotlib. Great for statistical plots.
- **Plotly**  Interactive, web-based plots and dashboards.
- **Bokeh**  Interactive and scalable visualizations for web.
- **Altair**  Declarative API for statistical visualization.
- **Pygal**  Simple & clean charts in SVG format.
- **Streamlit Charts**  Create charts easily in Streamlit apps.

💡 Best Practice

- ✓ Start with **Matplotlib**, then explore others.
- ✓ Use **Seaborn** for statistical analysis.
- ✓ Use **Plotly/Bokeh** for interactivity.
- ✓ Keep visuals simple and meaningful.
- ✓ Choose the right tool for the right use case.

🎯 Why Use Them?

- Understand patterns and trends easily
- Improve data storytelling
- Create reports, dashboards & analytics
- Support **interactive & real-time** visuals
- Useful in Data Science, BI & ML projects

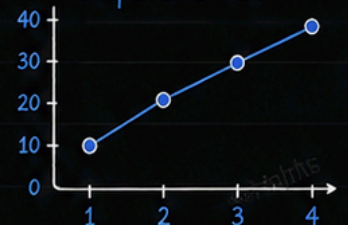
💡 When to Use What?

Library	Best For
Matplotlib	General purpose, static plots
Seaborn	Statistical & attractive plots
Plotly	Interactive dashboards
Bokeh	Large/real-time web visualizations
Altair	Quick statistical visualizations
Pygal	Lightweight charts (SVG)
Streamlit Charts	Data apps & dashboards

Example (Matplotlib)

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [10, 20, 30, 40]
plt.plot(x, y, marker='o')
plt.title('Simple Line Plot')
plt.show()
```

Simple Line Plot



★ **Interview Tip:** Be ready to explain the differences, use cases and when to choose each library.

Machine Learning Libraries



★ What?

- ML libraries provide ready-to-use algorithms and tools to build, train, evaluate and deploy models efficiently.
- They save time, improve accuracy and handle complex computations.

🎯 Why Use Them?

- Pre-built, optimized algorithms
- Faster development & experimentation
- Scalable for large datasets
- Better model performance
- Active community & good documentation

📖 Popular ML Libraries

★	Scikit-learn	General purpose ML library for classification, regression, clustering, preprocessing.
📊	XGBoost	Extreme Gradient Boosting. High performance & scalable.
⚡	LightGBM	Gradient boosting framework. Fast & memory efficient.
🐱	CatBoost	Gradient boosting with excellent handling of categorical data.
📈	Statsmodels	Statistical modeling and hypothesis testing.
⚙️	Feature Engine	Feature engineering & transformation made easy.

⚖️ Library Comparison

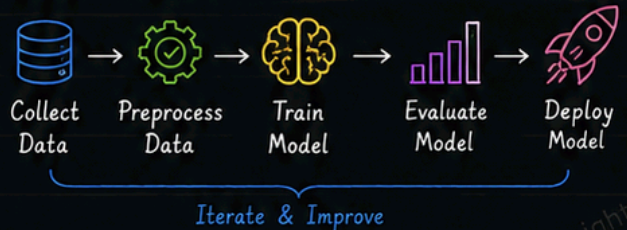
Library	Best For	Speed	Ease of Use
Scikit-learn	General ML Tasks	★★★★☆	★★★★★
XGBoost	Structured Data, Competitions	★★★★★	★★★★☆
LightGBM	Large Datasets, High Performance	★★★★★	★★★★☆
CatBoost	Categorical Data, Robust Models	★★★★☆	★★★★★
Statsmodels	Statistical Analysis, Econometrics	★★★★☆	★★★★☆
Feature Engine	Feature Engineering	★★★★★	★★★★★

</> Example (Scikit-learn)

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

Build and train a simple classification model

⚙️ ML Workflow



✅ Best Practice

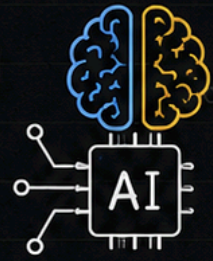
- Start with **Scikit-learn** for most use cases.
- Use **XGBoost** / **LightGBM** for tabular data with millions of rows.
- Always **validate**, **tune** and **iterate**!



💡 Interview Tip

- Know the strengths and use cases of each library.
- Understand **bias-variance tradeoff** and **overfitting**.

AI & Deep Learning Libraries



★ What?

- **AI & Deep Learning** libraries provide tools to build intelligent systems that can learn from data and make predictions.
- They power applications in computer vision, NLP, recommendation systems, chatbots, automation and more.

🎯 Why Use Them?

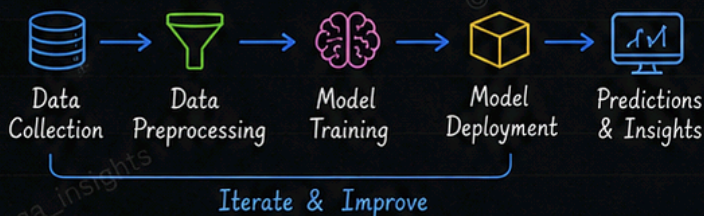
- High performance & scalability
- Pre-built models & components
- GPU/TPU acceleration support
- Active open-source communities
- Faster development & better results

📖 Popular AI & Deep Learning Libraries



Library	Best For	Key Features	Use Cases
TensorFlow	Deep Learning (End-to-End)	Flexible, scalable, production-ready, supports TF Serving, TFLite	Model training, deployment, Mobile/Edge AI
Keras	High-level Neural Networks	Simple & user-friendly API, runs on top of TF	Rapid prototyping, Neural network models
PyTorch	Research & Deep Learning	Dynamic computation graph, easy debugging	Research, NLP, CV, GANs, Reinforcement Learning
Hugging Face Transformers	NLP & Pre-trained Models	Thousands of pre-trained models, easy fine-tuning	Text generation, Translation, Sentiment analysis, QA
LangChain	LLM Apps & Agents	Build LLM apps, chains, agents, memory & tools integration	Chatbots, RAG, Agents, Document Q&A
OpenCV	Computer Vision	Image/Video processing, CV algorithms, real-time	Object detection, Face recognition, Tracking
MediaPipe	Vision & ML Pipelines	Pre-built ML pipelines for vision, face, pose, hands	Pose estimation, Hand tracking, Face mesh, AR apps

⚙️ How They Work Together



Example (TensorFlow + Keras)

```

import tensorflow as tf
from tensorflow import keras
model = keras.Sequential([
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam',
              loss='binary_crossentropy')
model.fit(X_train, y_train, epochs=10)
  
```

✅ Best Practices

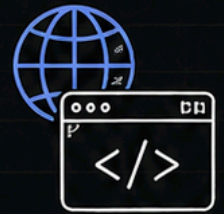
- Start simple → Try Keras / Transformers first.
- Use pre-trained models to save time.
- Always preprocess data correctly.
- Track experiments & versions.
- Validate, test & monitor your models.



💡 Interview Tip

Know the differences between TensorFlow, PyTorch and Keras. Understand concepts like overfitting, fine-tuning, transfer learning, embeddings and model deployment.

Web Development Libraries



★ What?

- **Web** libraries help build web applications, APIs and services using Python.
- They handle routing, templating, requests, responses, middleware and more.
- From simple apps to enterprise systems – Python has the right tools!

🎯 Why Use Them?

- Faster development with high productivity
- Scalable and secure web applications
- Great community and documentation
- Supports APIs, databases, authentication
- Used in startups to large-scale products

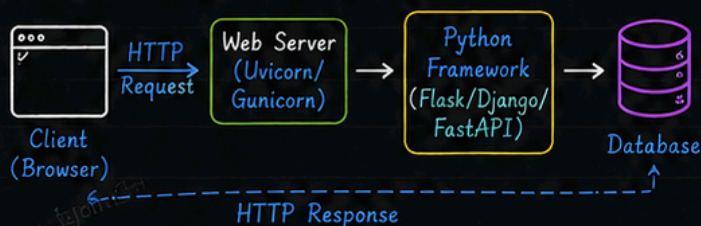
📖 Popular Web Development Libraries

- 1 **Flask** Lightweight micro web framework. Simple & flexible.
- 2 **Django** Full-featured web framework for rapid development.
- 3 **FastAPI** Modern, fast framework for building APIs with Python 3.7+ type hints.
- 4 **Jinja2** Powerful templating engine for HTML generation.
- 5 **Uvicorn** ASGI server for running async Python web apps.
- 6 **Gunicorn** WSGI HTTP server for running Python web applications.
- 7 **Django REST Framework (DRF)** Powerful toolkit for building Web APIs in Django.

⚖️ Library Comparison

Library	Type	Best For	Performance	Ease of Use
Flask	Micro Framework	Small Apps, APIs	★★★★☆	★★★★★
Django	Full Stack Framework	Large Apps, Enterprise	★★★★☆	★★★★☆
FastAPI	API Framework	APIs, Microservices, Async Apps	★★★★★	★★★★☆
Jinja2	Template Engine	HTML Templates in Python	★★★★★	★★★★★
Uvicorn	ASGI Server	Async Python Applications	★★★★★	★★★★☆
Gunicorn	WSGI Server	Production Deployments	★★★★☆	★★★★☆
DRF	API Toolkit (Django)	REST APIs in Django	★★★★☆	★★★★★

⚙️ How They Work Together



Example (Flask)

```

from flask import Flask
app = Flask(__name__)
@app.route('/')
def home():
    return 'Hello, Web World!'
if __name__ == '__main__':
    app.run(debug=True)
  
```

Run the app locally

✓ Best Practices

- Use **FastAPI** for high-performance APIs.
- Use **Django** for full-featured applications.
- Keep templates separate using **Jinja2**.
- Use **Gunicorn/Uvicorn** for production.



Interview Tip

Know the difference between **WSGI** and **ASGI**. Understand when to use **Flask** vs **Django** vs **FastAPI**.

API, Networking & Web Scrapping Libraries



★ What?

- Libraries to send **HTTP requests**, work with APIs, handle network operations and scrape data from websites.
- Useful for automation, testing, data collection, monitoring and integrations.
- Support sync, async, parsing and browser automation.



🎯 Why Use Them?

- Communicate with REST/GraphQL APIs
- Collect data from websites efficiently
- Handle authentication, headers, cookies
- Automate browsers for dynamic content
- Save time & build powerful integrations



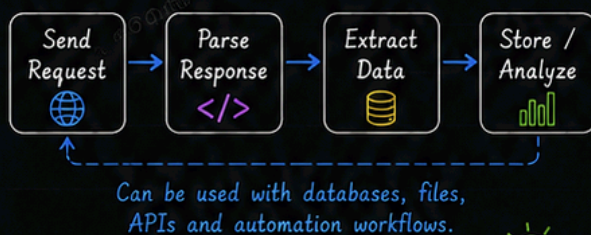
📖 Popular Libraries

1	Requests	Simple & user-friendly HTTP library.	🌐
2	HTTPX	Modern HTTP client with sync & async.	↔️
3	Aiohttp	Asynchronous HTTP client/server.	⚡
4	BeautifulSoup	Parse HTML/XML and extract data.	</>
5	Scrapy	Powerful web scraping framework.	🕷️
6	Selenium	Automate browsers (WebDriver).	🖥️
7	Playwright	Modern browser automation tool.	👤

⚖️ Library Comparison

Library	Type	Best For	Async Support	Ease of Use
Requests	HTTP Client	Simple API calls	✗	★★★★★
HTTPX	HTTP Client	Modern APIs, High perf.	✓	★★★★☆
Aiohttp	Async HTTP	High concurrency & async apps	✓	★★★★☆
BeautifulSoup	Parser	HTML/XML Parsing	✗	★★★★★
Scrapy	Scraping Framework	Large scale Web Scraping	✓	★★★★☆
Selenium	Browser Automation	Web UI Testing, Automation	✗	★★★★☆
Playwright	Browser Automation	Modern Web Automation	✓	★★★★☆

⚙️ How They Work Together



Example (Requests + BeautifulSoup)

```

import requests
from bs4 import BeautifulSoup
url = 'https://example.com'
resp = requests.get(url)
soup = BeautifulSoup(resp.text, 'html.parser')
print(soup.title.text)
  
```

Fetch page and extract title

✓ Best Practices

- Respect **robots.txt** and website terms.
- Use **timeouts**, **retries** and handle exceptions.
- For dynamic sites use **Selenium** / **Playwright**.
- Use **async** libraries for high performance.
- Store data in structured format (**CSV/DB**).

💡 Interview Tip

- Know the difference between **Requests**, **HTTPX** and **Aiohttp**.
- When to use **Scrapy** vs **BeautifulSoup**?
- Selenium** vs **Playwright** - key differences.
- How to handle **cookies**, **headers**, **proxies**?

Database Libraries



★ What?

- **Python** database libraries help connect, interact and manage different databases efficiently.
- Used for CRUD operations, ORM, migrations, caching and data modeling.
- Work with **SQL**, **NoSQL** and in-memory datastores.

🎯 Why Use Them?

- Simplify database connectivity and queries
- Support ORM, migrations and schema management
- Secure, fast and reliable data access
- Scalable for web apps, APIs and microservices
- Work with multiple database engines seamlessly

📖 Popular Database Libraries

1. **SQLAlchemy** Powerful ORM and SQL toolkit for relational databases.
2. **SQLite3** Built-in lightweight database engine. Serverless & file-based.
3. **Psycopg2** PostgreSQL adapter for Python. Fast and reliable.
4. **PyMySQL** Pure Python MySQL client library.
5. **PyMongo** Official Python driver for MongoDB (NoSQL).
6. **Redis-py** Python client for Redis. Great for caching & queues.
7. **Alembic** Database migration tool for SQLAlchemy.

⚖️ Library Comparison

Library	Type	Best For	DB Support
SQLAlchemy	ORM / Toolkit	Complex apps, ORM, queries	Multi-DB (SQL)
SQLite3	DB Engine	Local dev, small apps	SQLite
Psycopg2	Driver	PostgreSQL apps	PostgreSQL
PyMySQL	Driver	MySQL applications	MySQL / MariaDB
PyMongo	Driver	NoSQL apps, document data	MongoDB
Redis-py	Client	Caching, sessions, queues, pub/sub	Redis
Alembic	Migration Tool	DB migrations, schema versioning	SQLAlchemy based DBs

⚙️ How They Work Together



Example (SQLAlchemy)

```

from sqlalchemy import create_engine, text
engine = create_engine("sqlite://example.db")
with engine.connect() as conn:
    result = conn.execute(text("SELECT * FROM users"))
    print(result.fetchall())
  
```

Connect, execute and fetch data

✅ Best Practices

- Use SQLAlchemy for complex projects.
- Close connections / use context managers.
- Use **connection pooling** for performance.
- Store DB credentials in environment variables.
- Always validate & sanitize inputs.

💡 Interview Tip

- Know difference between ORM (SQLAlchemy) and Raw SQL.
- Understand **ACID** properties and **Transactions**.
- When to use **SQL** vs **NoSQL** databases?

Automation, DevOps & Cloud Libraries



★ What?

- **Libraries** that help automate infrastructure, deployments, tasks and cloud operations.
- Enable remote execution, container management, cloud services and scheduling jobs.
- Essential for DevOps, CI/CD, automation and cloud-native apps.

☺ Why Use Them?

- Automate infrastructure and deployments
- Secure remote connections & file transfers
- Manage containers and Kubernetes resources
- Interact with cloud services (AWS, etc.)
- Schedule and run background tasks reliably
- Improve scalability and operational efficiency

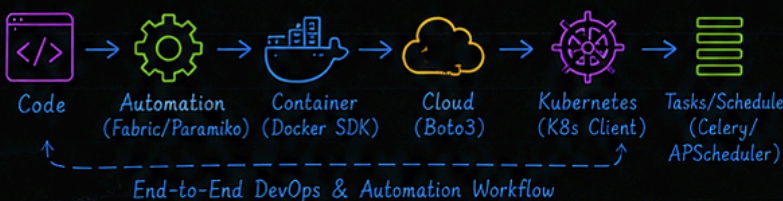
📖 Popular Libraries

- 1 **Fabric** Simplifies SSH operations and remote task execution.
- 2 **Paramiko** Low-level SSH library for secure connections.
- 3 **Docker SDK** Interact with Docker Engine via Python.
- 4 **Boto3** AWS SDK for Python to manage cloud services.
- 5 **Kubernetes Python Client** Manage Kubernetes clusters and resources.
- 6 **Celery** Distributed task queue for asynchronous processing.
- 7 **APScheduler** Job scheduling library for Python applications.

⚖ Library Comparison

Library	Type	Best For	Key Feature
Fabric	Automation (SSH)	Remote task automation	Simple & high-level SSH tasks
Paramiko	SSH Library	Low-level SSH operations	Secure connections & SFTP
Docker SDK	Container Management	Docker automation from Python	Full Docker Engine control
Boto3	Cloud SDK (AWS)	AWS services automation	Complete AWS service access
Kubernetes Python Client	K8s Client	Kubernetes resource management	Create, read, update K8s resources
Celery	Task Queue	Background tasks & workers	Distributed & scalable tasks
APScheduler	Scheduler	Scheduled jobs in Python apps	Cron-like scheduling in Python

⚙ How They Work Together



Example (Boto3 - List S3 Buckets)

```
import boto3
s3 = boto3.client('s3')
response = s3.list_buckets()
for bucket in response['Buckets']:
    print(bucket['Name'])
```

Connect to AWS S3 and list all buckets

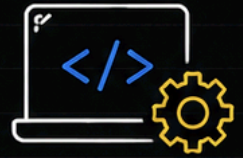
✓ Best Practices

- Store credentials in environment variables or use IAM roles!
- Use secrets managers (AWS Secrets Manager, HashiCorp Vault) for sensitive data.
- Handle exceptions and retries for network calls.
- Use **Celery** with **Redis/RabbitMQ** as broker.
- Automate with **CI/CD** for reliable deployments.

💡 Interview Tip

- Know differences between **Fabric** and **Paramiko**.
- Understand **Boto3** vs other AWS SDKs.
- How does **Celery** work? (Broker, Worker, Result Backend)
- When to use **Docker SDK** vs CLI?
- How to schedule jobs with **APScheduler**?

Developer Productivity Libraries



★ What?

- Libraries that **boost developer productivity**, improve code quality and simplify tasks.
- Help in validation, logging, CLI building, configuration, formatting, testing data and more.
- Save time, reduce boilerplate and write better Python code.

🔭 Why Use Them?

- Write cleaner, more readable and maintainable code
- Speed up development with smart utilities
- Better CLI, logging, validation and formatting
- Environment & configuration management
- Improve user experience in terminals & apps

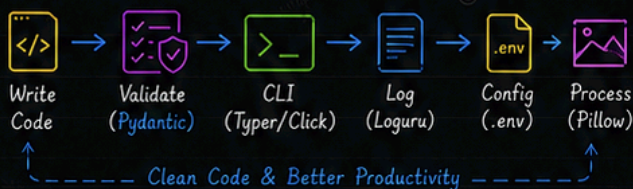
📖 Popular Productivity Libraries

- Pydantic** Data validation using type hints. Fast and reliable.
- Rich** Beautiful terminal output, tables, progress bars & more.
- Typer** Modern CLI framework based on type hints. Easy & fast.
- Click** Powerful CLI creation library for Python.
- Loguru** Simple, powerful and flexible logging made easy.
- python-dotenv** Load environment variables from .env files.
- Pillow** Image processing & manipulation for Python.
- Faker** Generate fake data for testing and development.
- Black** Code formatter for consistent and clean code.
- Ruff** Blazing-fast linter & formatter. All-in-one tool.

⚖️ Library Comparison

Library	Type	Best For	Key Feature
Pydantic	Validation	Data models, API schemas	Type based validation
Rich	Terminal UI	CLI apps, logs, dashboards	Beautiful & interactive output
Typer	CLI Framework	Modern CLI apps	Type hints + auto help
Click	CLI Framework	Custom CLIs & automations	Flexible & extensible
Loguru	Logging	Logging in apps	Simple syntax, powerful logs
python-dotenv	Config	Env variables management	Load .env effortlessly
Pillow	Image	Image processing, thumbnails, edits	Easy image manipulation
Faker	Testing Data	Generate fake test data	Many locales & providers
Black	Formatter	Code formatting	PEP8 compliant formats code
Ruff	Linter/Formatter	Linting + formatting	Extremely fast, all-in-one

⚙️ How They Work Together



Example (Pydantic + python-dotenv + Typer)

```

from pydantic import BaseModel
from dotenv import load_dotenv
import typer

load_dotenv() # Load .env file

class Settings(BaseModel):
    app_name: str
    debug: bool

app = typer.Typer()
@app.command()
def hello():
    s = Settings()
    typer.echo(f"Hello from {s.app_name}, debug={s.debug}")
  
```

Fast CLI with validation & config

✓ Best Practices

- Use **Pydantic** for data validation.
- Use **Rich** for better terminal UX.
- Store secrets in **.env**, never hardcode.
- Format with **Black** and lint with **Ruff**.
- Use **Faker** for realistic test data.

💡 Interview Tip

- Why use **Pydantic**? → Type safety & data validation.
- Black** vs **Ruff**? → Black formats, Ruff lints + formats.
- Typer** vs **Click**? → Typer is modern, Click is mature.
- When use **Loguru**? → When you want simple yet powerful logging.

Python Libraries Cheat Sheet (Part 1)



Data & AI Ecosystem

#	Library	What It Does	Key Features	When to Use	Icon
1	NumPy	Core library for numerical computing in Python.	- N-dimensional arrays - Fast math operations	Array ops, linear algebra, scientific computing	
2	Pandas	Data analysis and manipulation tool.	- DataFrames - Data cleaning, filtering	Data analysis, ETL, reporting	
3	Polars	Blazing-fast DataFrame library (Rust based).	- High performance - Lazy execution	Large datasets, fast analytics	
4	Matplotlib	Fundamental plotting library.	2D plots - Highly customizable	Basic charts, custom visualizations	
5	Seaborn	Statistical data visualization built on Matplotlib.	- Attractive themes - Statistical plots	EDA, correlation, distribution plots	
6	Plotly	Interactive and dynamic visualizations.	- Interactive charts 3D plots, dashboards	Web apps, dashboards, interactive reports	
7	Scikit-learn	ML algorithms and tools for data mining.	- Classification - Regression, clustering	ML models, preprocessing, evaluation	
8	TensorFlow	End-to-end open source ML platform.	- Deep learning - Production ready	Deep learning models, deployment	
9	PyTorch	Deep learning library by Meta.	- Dynamic graphs - Research friendly	Deep learning research & production	
10	Keras	High-level neural networks API.	- Simple & fast - Runs on TF, PyTorch	Rapid prototyping of deep learning	
11	OpenCV	Computer vision and image processing.	- Image/video processing - Face detection, filters	CV tasks, real-time applications	
12	Transformers	NLP models from Hugging Face.	- Pretrained models - NLP tasks & pipelines	Text generation, translation, summarization	
13	LangChain	Build LLM applications with orchestration.	- Chains, tools, agents - Integrate LLMs easily	LLM apps, chatbots, RAG workflows	



Data & AI Workflow



Example (Pandas + Matplotlib)

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('data.csv')
df.head()
df['sales'].plot(kind='line', title='Sales Trend')
plt.show()
```

Quick EDA with just a few lines!



Best Practices

- Use **Pandas/Polars** for data wrangling at scale.
- Visualize early to understand your data.
- Start simple with **Scikit-learn**, scale with DL tools.
- Leverage pretrained models (**Transformers**) to save time.
- Use **virtual environments** for dependency isolation.



Interview Tip

- Difference between **NumPy** array and **Pandas DataFrame**?
- When to use **Polars** over **Pandas**?
- Scikit-learn** vs **TensorFlow/PyTorch**?
- What is **Hugging Face Transformers** used for?
- Explain **LangChain** components (LLM, Chain, Agent, Tool).



Python Libraries Cheat Sheet (Part 2)

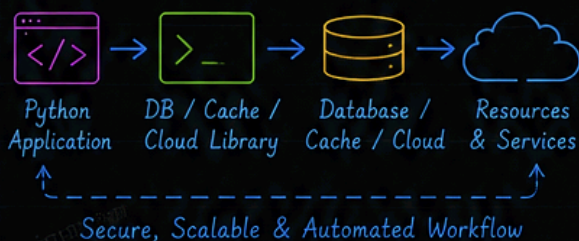


Database & DevOps Ecosystem

#	Library	What It Does	Key Features	When to Use	Icon
1	SQLAlchemy	Powerful ORM and SQL toolkit for Python.	- ORM + Core SQL - Migrations support - Connection pooling	Complex DB apps, ORM models, raw SQL, migrations	
2	SQLite3	Built-in lightweight SQL database engine.	- Zero configuration - Serverless - File-based DB	Local dev, small apps, embedded databases	
3	PyMySQL	Pure Python MySQL client library.	- MySQL client - Simple & clean API - Cursor support	Connect Python apps to MySQL databases	
4	Psycopg2	PostgreSQL adapter for Python.	- Fast & reliable - Advanced features - Prepared statements	Connect to PostgreSQL databases	
5	PyMongo	Official MongoDB driver for Python.	- Async support - Aggregation pipeline - GridFS support	Work with MongoDB (NoSQL) databases	
6	Redis-py	Python client for Redis key-value store.	- Cache, pub/sub - Pipelines, transactions - Cluster & Sentinel	Caching, sessions, queues, real-time data	
7	Docker SDK	Interact with Docker Engine using Python.	- Manage containers - Images, networks, volumes	Automate Docker workflows in Python apps	
8	Boto3	AWS SDK for Python to manage services.	- Low-level & resource interfaces - S3, EC2, IAM, etc.	Build & automate solutions on AWS	
9	Kubernetes Client	Python client for Kubernetes API.	- Manage K8s resources - Pods, Deployments, Services, ConfigMaps	Automate & manage Kubernetes clusters from Python	



How They Work Together



Example (PyMongo + Redis)

```
from pymongo import MongoClient
import redis
client = MongoClient("mongodb://localhost:27017")
db = client["mydb"]
col = db["users"]
cache = redis.Redis(host='localhost', port=6379)
data = col.find_one({"name": "Alice"})
cache.set("user:alice", str(data)) # cache result
print(data)
```

Store in MongoDB and cache in Redis for faster access!



Best Practices

- Use ORM (SQLAlchemy) for maintainable code.
- Always close connections / use context managers.
- Use connection pooling for production apps.
- Store secrets in .env or secret managers.
- Leverage caching (Redis) to reduce DB load.



Interview Tip

- Difference between ORM (SQLAlchemy) and raw SQL?
- When to use MongoDB vs PostgreSQL?
- How does Redis improve application performance?
- How to connect Python to AWS using Boto3?

